

lys_instr: A Python Package for Automating Scientific Measurements

Ziqian Wang^{1,2}, Hidenori Tsuji², Toshiya Shiratori³, and Asuka Nakamura^{2,3}

¹ Institutes of Innovation for Future Society, Nagoya University, Japan^{ROR} ² RIKEN Center for Emergent Matter Science, Japan^{ROR} ³ Department of Applied Physics, The University of Tokyo, Japan^{ROR}  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Modern experiments increasingly demand automation frameworks that coordinate diverse scientific instruments while remaining flexible and customizable. Developing custom measurement systems, however, often requires explicit management of low-level device communication and concurrency, resulting in substantial development overhead. We present `lys_instr`, a Python package that addresses these challenges through an object-oriented, three-layer architecture for instrument control, workflow coordination, and GUI construction. It enables researchers to construct responsive, asynchronous measurement systems by reusing instrument abstractions, workflow logic, and GUI components across hardware configurations. Integrated with the `lys` platform (Nakamura, 2023), `lys_instr` provides a common environment for experiment control, data acquisition, and multidimensional visualization.

Statement of need

Modern scientific research often relies on systematic measurements to elucidate physical phenomena. Such experiments may span wide parameter spaces, require long acquisition times, or coordinate multiple instruments, making efficient automation increasingly valuable. Measurement workflows may also incorporate informatics-driven, condition-based optimization, further motivating reusable and interoperable programmatic interfaces.

However, building such a measurement system remains time-consuming for researchers. At the low level, instrument-control implementations must accommodate diverse communication protocols (e.g., TCP/IP, VISA, and serial), often tying higher-level code to device-specific details and limiting device interchangeability and reuse across systems. At the high level, coordinating workflows that combine conditional logic, iterative processes, and advanced algorithms across multiple libraries frequently leads to redundant implementations, reducing development efficiency. For example, capturing temperature-dependent images with a camera and acquiring temperature-dependent spectra share the same general workflow: iterative temperature adjustment followed by data acquisition. Nevertheless, this logic is often implemented separately for different experiments. Moreover, for long or complex measurements, graphical user interfaces (GUIs) provide real-time visualization, device-state monitoring, and support for manual intervention. Implementing responsive GUIs for these low- and high-level functions, however, requires familiarity with GUI frameworks and event-handling mechanisms. These requirements impose substantial development overhead and highlight the need for a control framework that balances architectural flexibility with reduced implementation complexity.

State of the field

Scientific instrument-control software spans environments with different priorities. LabVIEW (National Instruments Corporation, 2024) and MATLAB's Instrument Control Toolbox (The MathWorks, Inc., 2024) integrate instrument communication, workflows, and GUI development. QCoDeS (Nielsen et al., 2025) and PyMeasure (PyMeasure Developers, 2024) provide instrument abstractions and experimental procedures. PyMoDAQ (Weber, 2021) combines actuator and detector plugins with dashboards and scan extensions, while Bluesky (Allan et al., 2019) separates hardware abstraction from reusable orchestration plans and optional GUI clients. Qudi (Binder et al., 2017) explicitly separates hardware, experiment logic, and GUI layers. Atomize (Melnikov et al., 2025) and fsc2 (Törring, 2019) support script-based experiments through reusable device operations. Together, these systems provide abstractions at the hardware, workflow, and GUI levels, but integrate them differently and target different workflows.

Among these systems, Qudi and PyMoDAQ provide the closest points of comparison for `lys_instr`. Qudi uses a three-layer architecture that decouples hardware, logic, and GUI modules, supporting reuse across hardware implementations. While its GUIs configure specific measurement toolchains, workflow composition remains specific to each toolchain rather than being provided through a shared graphical interface for composing and reorganizing nested measurement operations. PyMoDAQ provides GUI-configurable multidimensional and sequential scans, but organizes its functionality around instrument plugins, control modules, and dashboard extensions rather than an explicit three-layer hierarchy. The distinction of `lys_instr` lies in combining these strengths: an explicit three-layer architecture and GUI-based composition of nested workflows. Its Scan GUI allows users to construct and reorganize nested scans that coordinate controllers, switches, and detector acquisition. Once users implement hardware communication against its controller and multidimensional-detector interfaces, the same workflow and GUI components can be reused across hardware configurations, while acquired data can be visualized and analyzed directly through `lys`. These package-level design choices motivated the development of `lys_instr` as a distinct package rather than as an extension of another control framework.

Software design

The three layers and their relationships are summarized in Figure 1. Each layer applies established object-oriented design patterns (Gamma et al., 1994), including *Template Method*, *Bridge*, *Composite*, *Observer*, and *Mediator*, to define how components are extended, composed, and connected.

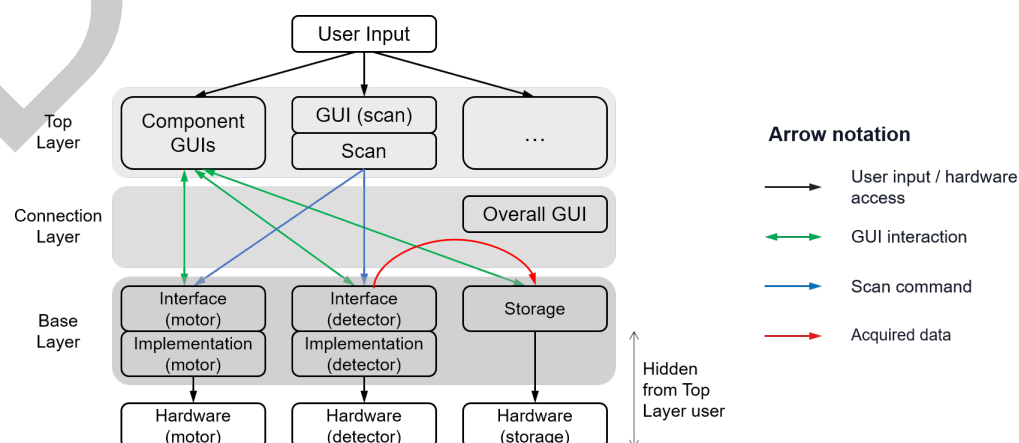


Figure 1: Schematic of the code architecture of `lys_instr`.

75 1. Base Layer: Instrument Abstraction

76 This layer defines standardized abstract interfaces for two principal roles in a control-and-
77 detect workflow: *controllers*, which adjust experimental parameters such as external fields,
78 temperature, or physical position, and *detectors*, which acquire data such as images or
79 spectra. Following the *Template Method* pattern, the base interfaces define the device
80 lifecycle, state monitoring, synchronization, and asynchronous execution, while subclasses
81 implement the primitive operations required for device-specific communication. Higher layers
82 interact with different instruments through the same methods, independently of their concrete
83 implementations. Because background monitoring and acquisition are managed by the
84 interfaces, other devices and GUIs can remain responsive, and device-specific subclasses
85 can be integrated directly into higher-level workflows without reimplementing concurrency
86 logic.

87 2. Top Layer: Workflow Coordination

88 This layer constructs hardware-independent workflows from the Base Layer interfaces. In a *scan*,
89 controllers vary experimental parameters while detectors acquire data at each step. Following
90 the *Bridge* pattern, scan execution depends on controller and detector abstractions rather than
91 concrete hardware implementations. The *Composite* pattern allows individual scan processes to
92 be nested into multi-parameter workflows while retaining a common execution model. Prebuilt
93 GUI components invoke the same abstract methods for direct device control and receive state
94 and data updates through event-driven signals, following the *Observer* pattern and avoiding
95 direct dependencies on concrete devices. Consequently, workflow and GUI logic can be reused
96 across different hardware configurations without rebuilding standard control and coordination
97 components.

98 3. Connection Layer: Control-System Assembly

99 This layer assembles components from the Base and Top Layers into an interactive control
100 system, including device interfaces, workflows, data storage, visualization, and GUI components.
101 Following the *Mediator* pattern, it centrally manages connections within and across layers,
102 linking GUI components to their corresponding interfaces and coordinating data flow without
103 direct dependencies among individual components. Concrete hardware details therefore remain
104 confined to the Base Layer, allowing system configurations and GUI layouts to be adapted
105 without application-level implementation of inter-device communication or thread coordination.
106 Prebuilt templates provide starting configurations that can be adapted to individual experimental
107 setups.

108 Together, these layers allow new instruments to be integrated through device-specific subclasses
109 while reusing asynchronous execution, workflow coordination, and GUI behavior. Workflows with
110 different coordination models can be supported through additional components or customized
111 interfaces.

112 **Example GUI configuration**

113 `lys_instr` allows users to assemble GUIs suited to individual experimental setups; [Figure 2](#)
114 shows one possible configuration. Here, the `lys_instr` window is embedded in the `lys`
115 platform, with Sector A for data storage, Sector B for detector control, and Sector C for
116 controller operation and scan configuration. Multidimensional, nested scan sequences can
117 be defined through the Scan tab, while `lys` tools in the outer window tabs allow on-the-fly
118 customization of acquired-data visualization.

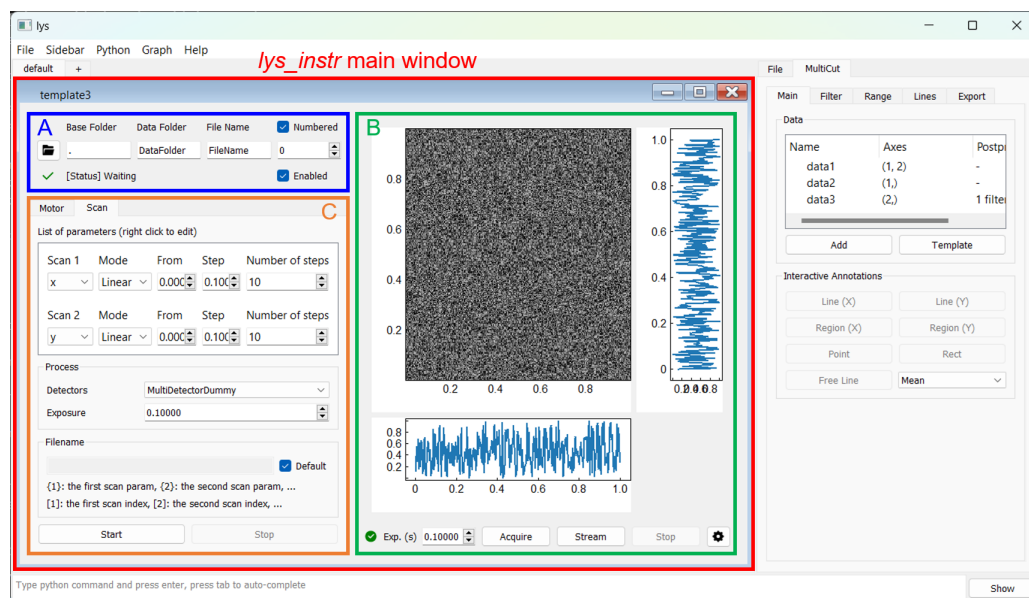


Figure 2: Example GUI configuration in `lys_instr`. The main window, embedded in the `lys` window, contains three sectors: storage (A), detector control (B), and controller operation and scan configuration (C). The Scan tab in (C) enables configuration of multidimensional, nested experimental workflows.

Research impact statement

`lys_instr` has been used in experiments reported in several peer-reviewed publications. It has been used to automate ultrafast electron diffraction (UED) and ultrafast transmission electron microscopy (UTEM) systems by coordinating experimental parameters and data acquisition in pump-probe experiments using ultrafast laser excitation and pulsed electron beams (Koga et al., 2024; Nakamura et al., 2020, 2021, 2022, 2023; Shimojima et al., 2021, 2023a, 2023b). It has also been used to control electromagnetic lenses and electron deflectors as part of complex microscopy workflows involving electron-beam precession (Hayashi et al., 2026; Shiratori et al., 2024).

The same interface-based workflows have been used to control transmission electron microscopes from multiple manufacturers at RIKEN Center for Emergent Matter Science and Nagoya University, demonstrating portability across hardware configurations. Integration with sister packages in the `lys` family, including `lys_em` (lys_em Developers, 2026) and `lys_fem` (lys_fem Developers, 2026), allows `lys_instr` components to participate in multi-instrument workflows while preserving modularity and extensibility.

AI usage disclosure

Generative AI tools provided debugging suggestions during the final stages of software development. The authors implemented and reviewed all changes. Core non-GUI behavior was tested with unit tests, while GUI and hardware workflows were validated with real instruments.

Acknowledgements

We acknowledge valuable comments from Takahiro Shimojima and Kyoko Ishizaka. This work was partially supported by Grant-in-Aid for Scientific Research (KAKENHI) Grants No. 21K13889, No. 26K17092, and No. 25K00057, and JST PRESTO Grant No. JPMJPR24JA.

References

- Allan, D., Caswell, T., Campbell, S., & Rakitin, M. (2019). Bluesky's Ahead: A multi-facility collaboration for an a la carte software project for data acquisition and management. *Synchrotron Radiation News*, 32(3), 19–22. <https://doi.org/10.1080/08940886.2019.1608121>
- Binder, J. M., Stark, A., Tomek, N., & others. (2017). Qudi: A modular python suite for experiment control and data processing. *SoftwareX*, 6, 85–90. <https://doi.org/10.1016/j.softx.2017.02.001>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. ISBN: 978-0201633610
- Hayashi, S., Han, D., Tsuji, H., Ishizaka, K., & Nakamura, A. (2026). Development of precession lorentz transmission electron microscopy. *Ultramicroscopy*, 280, 114276. <https://doi.org/10.1016/j.ultramic.2025.114276>
- Koga, J., Chiashi, Y., Nakamura, A., Akiba, T., Takahashi, H., Shimojima, T., Ishiwata, S., & Ishizaka, K. (2024). Unusual photoinduced crystal structure dynamics in TaTe₂ with double zigzag chain superstructure. *Applied Physics Express*, 17(4), 042007. <https://doi.org/10.35848/1882-0786/ad3b61>
- lys_em Developers. (2026). *Lys_em: Python package for electron microscopy control and data analysis*. https://github.com/a-tock/lys_em
- lys_fem Developers. (2026). *Lys_fem: Python package for finite-element modeling and experiment integration*. https://github.com/lys-devel/lys_fem
- Melnikov, A., Vedkal, A., Ishchenko, A., & Veber, S. (2025). Atomize: A modular software for control and automation of scientific and industrial instruments. *Journal of Open Research Software*, 13, 26. <https://doi.org/10.5334/jors.594>
- Nakamura, A. (2023). lys: Interactive Multi-Dimensional Data Analysis and Visualization Platform. *Journal of Open Source Software*, 8(92), 5869. <https://doi.org/10.21105/joss.05869>
- Nakamura, A., Shimojima, T., Chiashi, Y., Kamitani, M., Sakai, H., Ishiwata, S., Li, H., & Ishizaka, K. (2020). Nanoscale Imaging of Unusual Photoacoustic Waves in Thin Flake VTe₂. *Nano Letters*, 20(7), 4932–4938. <https://doi.org/10.1021/acs.nanolett.0c01006>
- Nakamura, A., Shimojima, T., & Ishizaka, K. (2021). Finite-element Simulation of Photoinduced Strain Dynamics in Silicon Thin Plates. *Structural Dynamics*, 8(2), 024103. <https://doi.org/10.1063/4.0000059>
- Nakamura, A., Shimojima, T., & Ishizaka, K. (2022). Visualizing Optically-Induced Strains by Five-Dimensional Ultrafast Electron Microscopy. *Faraday Discussions*, 237, 27–39. <https://doi.org/10.1039/D2FD00062H>
- Nakamura, A., Shimojima, T., & Ishizaka, K. (2023). Characterizing an Optically Induced Sub-micrometer Gigahertz Acoustic Wave in a Silicon Thin Plate. *Nano Letters*, 23(7), 2490–2495. <https://doi.org/10.1021/acs.nanolett.2c03938>
- National Instruments Corporation. (2024). *LabVIEW*. National Instruments Corporation. <https://www.ni.com/en-us/shop/labview.html>
- Nielsen, J. H., Astafev, M., Nielsen, W. H. P., & others. (2025). *QCoDeS: Modular data acquisition framework* (Version v0.54.2). <https://doi.org/10.5281/zenodo.17459861>
- PyMeasure Developers. (2024). *PyMeasure: Scientific measurement library for instruments, experiments, and live-plotting*. (Version 0.14.0). Zenodo. <https://doi.org/10.5281/zenodo.11241567>

- 188 Shimojima, T., Nakamura, A., & Ishizaka, K. (2023a). Development of Five-Dimensional
189 Scanning Transmission Electron Microscopy. *Review of Scientific Instruments*, 94(2),
190 023705. <https://doi.org/10.1063/5.0106517>
- 191 Shimojima, T., Nakamura, A., & Ishizaka, K. (2023b). Development and Applications
192 of Ultrafast Transmission Electron Microscopy. *Microscopy*, 72(4), 287–298. <https://doi.org/10.1093/jmicro/dfad021>
193
- 194 Shimojima, T., Nakamura, A., Yu, X., Karube, K., Taguchi, Y., Tokura, Y., & Ishizaka, K.
195 (2021). Nano-to-Micro Spatiotemporal Imaging of Magnetic Skyrmion's Life Cycle. *Science*
196 *Advances*, 7(25), eabg1322. <https://doi.org/10.1126/sciadv.abg1322>
- 197 Shiratori, T., Koga, J., Shimojima, T., Ishizaka, K., & Nakamura, A. (2024). Development of
198 ultrafast four-dimensional precession electron diffraction. *Ultramicroscopy*, 267, 114064.
199 <https://doi.org/10.1016/j.ultramic.2024.114064>
- 200 The MathWorks, Inc. (2024). *MATLAB instrument control toolbox*. The MathWorks, Inc.
201 <https://www.mathworks.com/products/instrument.html>
- 202 Törring, J. T. (2019). *fsc2: A program for controlling spectrometers*. [https://users.physik.fu-](https://users.physik.fu-berlin.de/~jtt/fsc2.phtml)
203 [berlin.de/~jtt/fsc2.phtml](https://users.physik.fu-berlin.de/~jtt/fsc2.phtml)
- 204 Weber, S. J. (2021). PyMoDAQ: An open-source python-based software for modular data
205 acquisition. *Review of Scientific Instruments*, 92(4), 045104. [https://doi.org/10.1063/5.](https://doi.org/10.1063/5.0032116)
206 [0032116](https://doi.org/10.1063/5.0032116)

DRAFT